Solution: Longest Substring with K Distinct Characters

Problem Statement

Given a string, find the length of the **longest substring** in it **with no more than K distinct characters.**

You can assume that K is less than or equal to the length of the given string.

Example 1:

```
Input: String="araaci", K=2
Output: 4
Explanation: The longest substring with no more than '2' distinct characters is "ar
aa".
```

Example 2:

```
Input: String="araaci", K=1
Output: 2
Explanation: The longest substring with no more than '1' distinct characters is "aa
".
```

Example 3:

```
Input: String="cbbebi", K=3
Output: 5
Explanation: The longest substrings with no more than '3' distinct characters are "
cbbeb" & "bbebi".
```

Solution

This problem follows the **Sliding Window** pattern, and we can use a similar dynamic sliding window strategy as discussed in Smallest Subarray with a Greater Sum. We can use a **HashMap** to remember the frequency of each character we have processed. Here is how we will solve this problem:

1. First, we will insert characters from the beginning of the string until we have K distinct characters in the **HashMap**.
2. These characters will constitute our sliding window. We are asked to find the longest such window having no more than K distinct characters. We will remember the length of this window as the

longest window so far.

3. After this, we will keep adding one character in the sliding window (i.e., slide the window ahead) in a stepwise fashion.
4. In each step, we will try to shrink the window from the beginning if the count of distinct characters in the **HashMap** is larger than K. We will shrink the window until we have no more than K distinct characters in the **HashMap**. This is needed as we intend to find the longest window.
5. While shrinking, we'll decrement the character's frequency going out of the window and remove it from the **HashMap** if its frequency becomes zero.
6. At the end of each step, we'll check if the current window length is the longest so far, and if so, remember its length.

Here is the visual representation of this algorithm for the Example-1:

Code

Here is what our algorithm will look like:

Java

```java
import java.util.*;

class Solution {
    public int findLength(String str, int k) {
        if (str == null || str.length() == 0 || str.length() < k)
            throw new IllegalArgumentException();

        int windowStart = 0, maxLength = 0;
        Map<Character, Integer> charFrequencyMap = new HashMap<>();
        // in the following loop we'll try to extend the range [windowStart, windowEnd]
        for (int windowEnd = 0; windowEnd < str.length(); windowEnd++) {
            char rightChar = str.charAt(windowEnd);
            charFrequencyMap.put(rightChar, charFrequencyMap.getOrDefault(rightChar, 0) + 1);
            // shrink the sliding window, until we are left with 'k' distinct characters in
            // the frequency map
            while (charFrequencyMap.size() > k) {
                char leftChar = str.charAt(windowStart);
                charFrequencyMap.put(leftChar, charFrequencyMap.get(leftChar) - 1);
                if (charFrequencyMap.get(leftChar) == 0) {
                    charFrequencyMap.remove(leftChar);
                }
                windowStart++; // shrink the window
            }
            // remember the maximum length so far
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The above algorithm's time complexity will be $O(N)$, where N is the number of characters in the input string. The outer `for` loop runs for all characters, and the inner `while` loop processes each character only once; therefore, the time complexity of the algorithm will be $O(N + N)$, which is asymptotically equivalent to $O(N)$.

Space Complexity

The algorithm's space complexity is $O(K)$, as we will be storing a maximum of $K + 1$ characters in the HashMap.

← Previous

Longest Substring With K Distinct Characters (Medium)

Next →

Fruits Into Baskets (Medium)

Mark as Completed